

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week4 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:8.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases Lab Objectives: To introduce students to test-driven development (TDD) using AI code generation tools.		Week4 - Monday

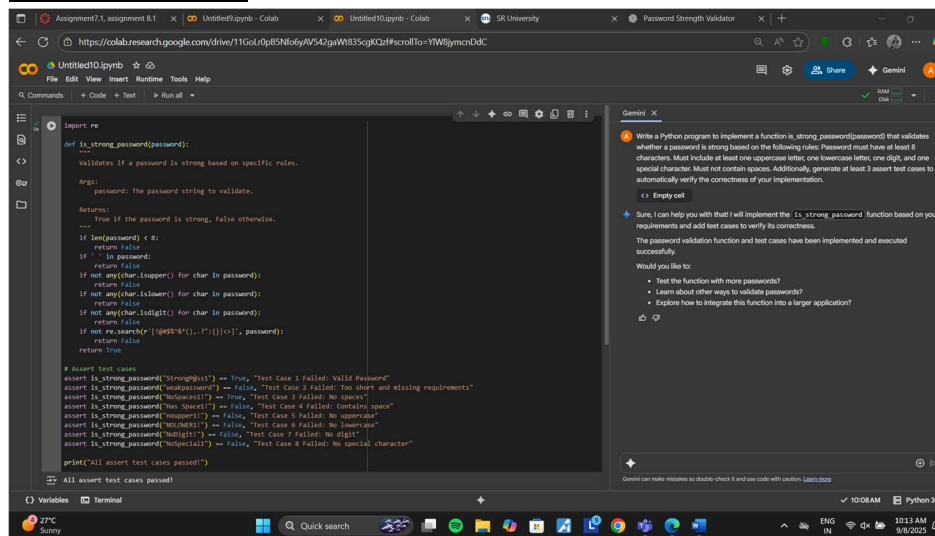
	• To enable the generation of test cases before writing code implementations. • To reinforce the importance of testing, validation, and error handling. • To encourage writing clean and reliable code based on AI-generated test expectations. Lab Outcomes (LOs): After completing this lab, students will be able to: • Use AI tools to write test cases for Python functions and classes. • Implement functions based on test cases in a test-first development style. • Use unittest or pytest to validate code correctness. • Analyze the completeness and coverage of AI-generated tests. • Compare AI-generated and manually written test cases for quality and logic Task Description #1 (Password Strength Validator – Apply AI in Security Context) • Task: Apply AI to generate at least 3 assert test cases for <code>is_strong_password(password)</code> and implement the validator function. • Requirements: ◦ Password must have at least 8 characters. ◦ Must include uppercase, lowercase, digit, and special character. ◦ Must not contain spaces. Example Assert Test Cases: <pre>assert is_strong_password("Abcd@123") == True assert is_strong_password("abcd123") == False assert is_strong_password("ABCD@1234") == True</pre> Expected Output #1: • Password validation logic passing all AI-generated test cases. <u>PROMPT—</u> Write a Python program to implement a function <code>is_strong_password(password)</code> that validates whether a password is strong based on the following rules: • Password must have at least 8 characters. • Must include at least one uppercase letter, one lowercase letter,	
--	---	--

one digit, and one special character.

- Must not contain spaces.

Additionally, generate at least **3 assert test cases** to automatically verify the correctness of your implementation.

FINAL OUPUT --



The screenshot shows a Jupyter Notebook interface with a code editor on the left and a terminal on the right. The code defines a function `is_strong_password(password)` that checks if a password is strong based on specific rules: it must be at least 8 characters long, contain at least one uppercase letter, one lowercase letter, one digit, and one special character. The function returns `True` if the password is strong and `False` otherwise. Below the function definition, there are several assert test cases for various passwords, including "Strong@123", "WeakPassword", "NoSpaces1", "No Space!", "NoSpecial", "NoLowercase", "NoDigit", and "NoSpecialChar". The terminal output shows that all test cases passed.

```
def is_strong_password(password):
    """
    Validates if a password is strong based on specific rules.
    Args:
        password: The password string to validate.
    Returns:
        True if the password is strong, False otherwise.
    """
    if len(password) < 8:
        return False
    if not password:
        return False
    if not any(char.isupper() for char in password):
        return False
    if not any(char.islower() for char in password):
        return False
    if not any(char.isdigit() for char in password):
        return False
    if not re.search(r'[!@#$%^&*()-+=~`~\|;:,./< >{}[]\~]', password):
        return False
    return True

# Assert test cases
assert is_strong_password("Strong@123") == True, "Test Case 1 Failed: Valid Password"
assert is_strong_password("WeakPassword") == False, "Test Case 2 Failed: Too short and missing requirements"
assert is_strong_password("NoSpaces1") == True, "Test Case 3 Failed: No Spaces"
assert is_strong_password("No Space!") == False, "Test Case 4 Failed: Contains space"
assert is_strong_password("NoSpecial") == False, "Test Case 5 Failed: No special char"
assert is_strong_password("NoLowercase") == False, "Test Case 6 Failed: No lowercase"
assert is_strong_password("NoDigit") == False, "Test Case 7 Failed: No digit"
assert is_strong_password("NoSpecialChar") == False, "Test Case 8 Failed: No special character"

print("All assert test cases passed!")
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.
- Requirements:
 - Classify numbers as Positive, Negative, or Zero.
 - Handle invalid inputs like strings and None.
 - Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

`assert classify_number(10) == "Positive"`

`assert classify_number(-5) == "Negative"`

`assert classify_number(0) == "Zero"`

Expected Output #2:

- Classification logic passing all assert tests.

PROMPT—

Write a Python program to implement a function `classify_number(n)` that classifies a given number as:

- "Positive" if the number is greater than 0.

- "Negative" if the number is less than 0.
- "Zero" if the number is equal to 0.

Requirements:

- Use **loops** where applicable.
- Handle invalid inputs such as strings and None (return "Invalid input").
- Include boundary conditions (-1, 0, 1).
- Generate at least **3 assert test cases** to automatically verify the correctness of your function.

FINAL OUPUT--

```

def classify_number(n):
    """
    Classifies a given number as Positive, Negative, or Zero.

    Args:
        n: The number to classify.

    Returns:
        "Positive" if n > 0, "Negative" if n < 0, "Zero" if n == 0.
        Returns "Invalid input" for non-numeric or None inputs.
    """
    # Handle invalid inputs
    if not isinstance(n, (int, float)):
        return "Invalid input"
    if n is None:
        return "Invalid input"

    # Classify the number
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"

    # Assert test cases
    assert classify_number(5) == "Positive", "Test Case 1 Failed: Positive number"
    assert classify_number(-10) == "Negative", "Test Case 2 Failed: Negative number"
    assert classify_number(0) == "Zero", "Test Case 3 Failed: Zero"
    assert classify_number(1) == "Positive", "Test Case 4 Failed: Boundary condition 1"
    assert classify_number(-1) == "Negative", "Test Case 5 Failed: Boundary condition -1"
    assert classify_number("abc") == "Invalid input", "Test Case 6 Failed: String input"
    assert classify_number(None) == "Invalid input", "Test Case 7 Failed: None input"
    assert classify_number(3.14) == "Positive", "Test Case 8 Failed: Float input"

    print("All assert test cases passed!")
  
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.
- Requirements:
 - Ignore case, spaces, and punctuation.
 - Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

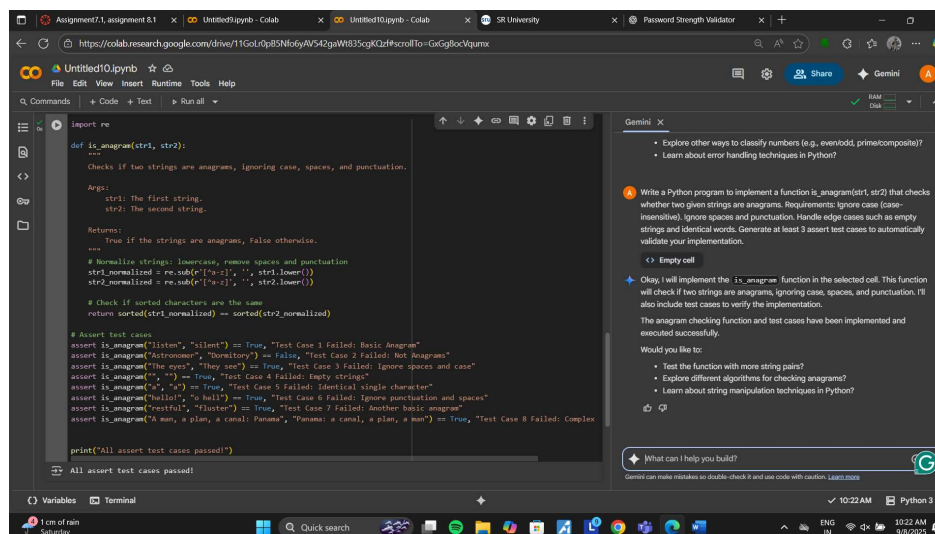
PROMPT --

Write a Python program to implement a function `is_anagram(str1, str2)` that checks whether two given strings are anagrams.

Requirements:

- Ignore case (case-insensitive).
- Ignore spaces and punctuation.
- Handle edge cases such as empty strings and identical words.
- Generate at least **3 assert test cases** to automatically validate your implementation.

FINAL OUTPUT --



The screenshot shows a Jupyter Notebook interface with a Python function `is_anagram` and its test cases. The function normalizes strings by converting them to lowercase, removing spaces and punctuation, and then checks if the sorted characters are the same. The test cases include assertions for basic anagrams, non-anagrams, empty strings, identical single characters, and strings with punctuation and spaces. The output shows that all test cases passed.

```
import re

def is_anagram(str1, str2):
    """
    Checks if two strings are anagrams, ignoring case, spaces, and punctuation.

    Args:
        str1: The first string.
        str2: The second string.

    Returns:
        True if the strings are anagrams, False otherwise.
    """
    # Normalize strings: lowercase, remove spaces and punctuation
    str1_normalized = re.sub(r'[\s\p{P}]', '', str1.lower())
    str2_normalized = re.sub(r'[\s\p{P}]', '', str2.lower())

    # Check if sorted characters are the same
    return sorted(str1_normalized) == sorted(str2_normalized)

# Assert test cases
assert is_anagram("listen", "silent") == True, "Test Case 1 Failed: Basic Anagram"
assert is_anagram("Astronomer", "Dormitory") == False, "Test Case 2 Failed: Not Anagrams"
assert is_anagram("the eyes", "they see") == True, "Test Case 3 Failed: Ignore spaces and case"
assert is_anagram("", "") == True, "Test Case 4 Failed: Empty strings"
assert is_anagram("a", "a") == True, "Test Case 5 Failed: Identical single character"
assert is_anagram("hello", "o hell") == True, "Test Case 6 Failed: Ignore punctuation and spaces"
assert is_anagram("craft", "flatter") == True, "Test Case 7 Failed: Another basic anagram"
assert is_anagram("A man, a plan, a canal: Panama", "Panama: a canal, a plan, a man") == True, "Test Case 8 Failed: Complex"

print("All assert test cases passed!")
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
- Methods:
 - `add_item(name, quantity)`
 - `remove_item(name, quantity)`
 - `get_stock(name)`

Example Assert Test Cases:

```
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.

PROMPT –

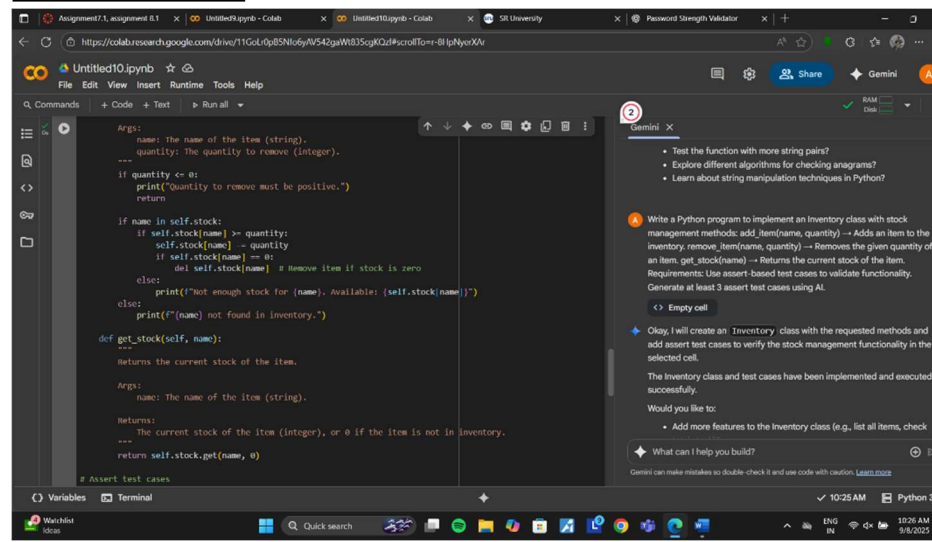
Write a Python program to implement an Inventory class with stock management methods:

- add_item(name, quantity) → Adds an item to the inventory.
- remove_item(name, quantity) → Removes the given quantity of an item.
- get_stock(name) → Returns the current stock of the item.

Requirements:

- Use assert-based test cases to validate functionality.
- Generate at least **3 assert test cases** using AI.

FINAL OUPUT –



```
Args:
  name: The name of the item (string).
  quantity: The quantity to remove (integer).
  ...
if quantity <= 0:
    print("Quantity to remove must be positive.")
    return

if name in self.stock:
    if self.stock[name] >= quantity:
        self.stock[name] -= quantity
        if self.stock[name] == 0:
            del self.stock[name] # Remove item if stock is zero
    else:
        print(f"not enough stock for {name}. Available: {self.stock[name]}")
    else:
        print(f"{name} not found in inventory.")

def get_stock(self, name):
    """
    Returns the current stock of the item.

    Args:
        name: The name of the item (string).

    Returns:
        The current stock of the item (integer), or 0 if the item is not in inventory.
    """
    return self.stock.get(name, 0)

# Assert test cases
```

Gemini

- Test the function with more string pairs?
- Explore different algorithms for checking anagrams?
- Learn about string manipulation techniques in Python?

Write a Python program to implement an Inventory class with stock management methods: add_item(name, quantity) → Adds an item to the inventory, remove_item(name, quantity) → Removes the given quantity of an item, get_stock(name) → Returns the current stock of the item. Requirements: Use assert-based test cases to validate functionality. Generate at least 3 assert test cases using AI.

Empty cell

Okay, I will create an Inventory class with the requested methods and add assert test cases to verify the stock management functionality in the selected cell.

The Inventory class and test cases have been implemented and executed successfully.

Would you like to:

- Add more features to the Inventory class (e.g., list all items, check

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
# Assert test cases
Inventory = Inventory()

# Test Case 1: Add item
Inventory.add_item("apple", 10)
assert Inventory.get_stock("apple") == 10, "Test Case 1 Failed: Add item"

# Test Case 2: Add more of the same item
Inventory.add_item("apple", 5)
assert Inventory.get_stock("apple") == 15, "Test Case 2 Failed: Add more of the same item"

# Test Case 3: Remove item
Inventory.remove_item("apple", 7)
assert Inventory.get_stock("apple") == 8, "Test Case 3 Failed: Remove item"

# Test Case 4: Remove more than available
Inventory.remove_item("apple", 10)
assert Inventory.get_stock("apple") == 8, "Test Case 4 Failed: Remove more than available" # Stock should remain 8

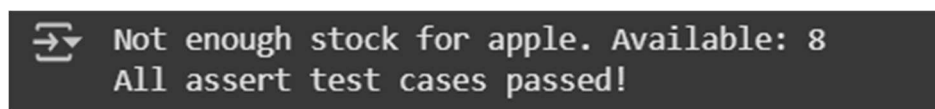
# Test Case 5: Remove all of an item
Inventory.remove_item("apple", 8)
assert Inventory.get_stock("apple") == 0, "Test Case 5 Failed: Remove all of an item"

# Test Case 6: Get stock of non-existent item
assert Inventory.get_stock("banana") == 0, "Test Case 6 Failed: Get stock of non-existent item"

# Test Case 7: Add another item
Inventory.add_item("banana", 20)
assert Inventory.get_stock("banana") == 20, "Test Case 7 Failed: Add another item"

print("All assert test cases passed!")
```

RESULT –



Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.
- Requirements:
 - Validate "MM/DD/YYYY" format.
 - Handle invalid dates.
 - Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

PROMPT –

Write a Python program to implement a function

`validate_and_format_date(date_str)` that validates and converts dates.

Requirements:

- Input must be in "MM/DD/YYYY" format.
- Validate whether the given date is valid (e.g., reject 02/30/2023).
- If valid, convert and return the date in "YYYY-MM-DD" format.
- If invalid, return "Invalid Date".
- Use AI to generate at least 3 assert test cases to verify correctness.

FINAL OUTPUT –

The screenshot shows a Google Colab notebook with the following code in a cell:

```
from datetime import datetime

def validate_and_format_date(date_str):
    """
    Validates if a date string is in "MM/DD/YYYY" format and is a valid date,
    then converts it to "YYYY-MM-DD" format.

    Args:
        date_str: The date string to validate and format.

    Returns:
        The date in "YYYY-MM-DD" format if valid, otherwise "Invalid Date".
    """
    try:
        # Attempt to parse the date string in MM/DD/YYYY format
        date_obj = datetime.strptime(date_str, "%m/%d/%Y")
        # If parsing is successful, format it to YYYY-MM-DD
        return date_obj.strftime("%Y-%m-%d")
    except ValueError:
        # If parsing fails (invalid format or invalid date), return "Invalid Date"
        return "Invalid Date"

# Assert test cases
assert validate_and_format_date("10/26/2023") == "2023-10-26", "Test Case 1 Failed: Valid Date"
assert validate_and_format_date("02/30/2023") == "Invalid Date", "Test Case 2 Failed: Invalid Date (day out of range)"
assert validate_and_format_date("13/01/2023") == "Invalid Date", "Test Case 3 Failed: Invalid Date (month out of range)"
assert validate_and_format_date("10-26-2023") == "Invalid Date", "Test Case 4 Failed: Invalid format"
assert validate_and_format_date("10/26/23") == "Invalid Date", "Test Case 5 Failed: Invalid Year Format"
assert validate_and_format_date("01/01/2000") == "2000-01-01", "Test Case 6 Failed: Valid Date - Start of Year"
assert validate_and_format_date("12/31/2017") == "2017-12-31", "Test Case 7 Failed: Valid Date - End of Year"

print("All assert test cases passed!")
```

The Gemini chat window on the right contains the following response:

Okay, I will implement the `validate_and_format_date` function in the selected cell. This function will validate if a date string is in the correct "MM/DD/YYYY" format and is a valid date. If valid, it will convert it to "YYYY-MM-DD" format, otherwise, it will return "Invalid Date". I will also add test cases.

The date validation and formatting function and test cases have been implemented and executed successfully.

Would you like to:

- Test the function with more date strings?
- Explore different date and time manipulation techniques in Python?
- Learn about handling timezones in Python?

✓ Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

Evaluation Criteria:

	<table><tr><th>Criteria</th><th>Max Marks</th></tr><tr><td>Task #1</td><td>0.5</td></tr><tr><td>Task #2</td><td>0.5</td></tr><tr><td>Task #3</td><td>0.5</td></tr><tr><td>Task #4</td><td>0.5</td></tr><tr><td>Task #5</td><td>0.5</td></tr><tr><td>Total</td><td>2.5 Marks</td></tr></table>	Criteria	Max Marks	Task #1	0.5	Task #2	0.5	Task #3	0.5	Task #4	0.5	Task #5	0.5	Total	2.5 Marks	
Criteria	Max Marks															
Task #1	0.5															
Task #2	0.5															
Task #3	0.5															
Task #4	0.5															
Task #5	0.5															
Total	2.5 Marks															